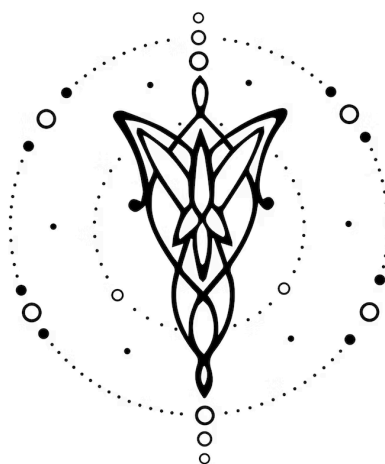




C+

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	4

1. Equipo

Nombre	Apellido	Legajo	E-mail
Emilio	Mitchell	64590	emitchell@itba.edu.ar
Lucia	Oliveto	64646	loliveto@itba.edu.ar
Tomás	Pietravallo	64288	tpietravallo@itba.edu.ar
Máximo	Wehncke	64018	mwehncke@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en el siguiente [repositorio](#).

3. Dominio

Proveer un nivel de abstracción mayor sobre el lenguaje C introduciendo clases al lenguaje. Las clases permiten contener variables de instancia y métodos de forma simple en comparación a la experiencia actual que provee el lenguaje. Para reducir la complejidad del proyecto, estas clases se transformarán a tipos abstractos de datos (ADTs) que serán los que proveerán el comportamiento en tiempo de ejecución.

La implementación satisfactoria del lenguaje extendería la practicidad y experiencia de desarrollo en el lenguaje C. Permitiendo fácilmente encapsular y modularizar código C.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrá crear una clase.
- (II). Se podrá instanciar una clase a través de su constructor.
- (III). Las clases podrán tener variables de instancia tanto públicas como privadas.
- (IV). Las clases podrán tener variables estáticas de tipo tanto públicas como privadas.
- (V). Las clases podrán tener métodos de instancia tanto públicos como privados.
- (VI). Las clases podrán tener métodos estáticos tanto públicos como privados.
- (VII). Se podrá acceder a las variables públicas de una clase.
- (VIII). Se podrán invocar los métodos públicos de una clase.
- (IX). Los métodos de una clase podrán ser sobrecargados según los tipos y cantidad de parámetros que reciban.
- (X). Se podrán crear clases que dependan de un tipo de dato genérico.
- (XI). Las clases se podrán retornar y pasar como parámetro.
- (XII). Las clases se podrán heredar de clases previamente creadas.
- (XIII). Habrá disponible un destructor para el manejo de memoria dinámica.
- (XIV). El compilador soportará los operadores aritmético-lógicos: '+', '-', '==', '&&' y '||', correspondientes a la suma, resta, igualdad, conjunción y disyunción.
- (XV). Los operadores punto y flecha serán soportados en sus usos estándar dentro del lenguaje C, y permitirán acceder y/o desreferenciar los miembros públicos de las clases.
- (XVI). Dentro de un método de instancia se podrán acceder a las variables y métodos de instancia mediante la palabra reservada 'this'.
- (XVII). Se podrá definir e invocar funciones definidas con un tipo de dato de retorno explícito, argumentos entre paréntesis y un cuerpo de función entre llaves; tal y como en el lenguaje C. A su vez, también soportará algunas funciones de la librería estándar como 'printf()'.
- (XVIII). Se permitirá usar la directiva '#include', para incorporar encabezados que definan prototipos de funciones y/o constantes.

- (XIX). Se permitirá la creación de estructuras de datos, cómo por ejemplo arreglos y structs.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que crea una clase con variables de instancia públicas y se acceden desde afuera.
- (II). Un programa que crea una clase con variables de instancia privadas y métodos “getters” públicos y accede a las variables mediante los getters desde afuera.
- (III). Un programa que instancia múltiples veces una clase que creó previamente.
- (IV). Un programa que crea una clase que utiliza int como tipo de dato genérico.
- (V). Un programa que crea una clase con dos métodos, donde ambos tienen el mismo nombre pero distintos parámetros. Luego se crea una instancia de la clase y se hace el llamado a ambos métodos, cada uno con sus distintos parámetros.
- (VI). Un programa que no declara clases.
- (VII). Un programa que al crear clases no explicita un constructor para las mismas.
- (VIII). Un programa que posee clases con constructores.
- (IX). Un programa que crea una clase con un método estático y luego lo llama desde afuera sin haber instanciado la clase.
- (X). Un programa que crea una clase que sólo contiene variables estáticas.
- (XI). Un programa que crea una clase y posee algún método o función que recibe como parámetro dicha clase. Lo mismo si la retorna.
- (XII). Un programa que crea una clase, y luego otra que hereda de ella.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa que instancia una clase que no fue creada previamente.
- (II). Un programa que accede a una variable privada de una clase instanciada.
- (III). Un programa que invoca un método privado de una clase instanciada.
- (IV). Un programa que invoca un método de una clase que no está definido en la misma.
- (V). Un programa que crea una clase cuyo constructor no tiene el mismo nombre que la clase.
- (VI). Un programa que explicita un retorno en el método constructor de una clase.
- (VII). Un programa que crea una función que recibe un parámetro del tipo de una clase que no fue creada previamente.
- (VIII). Un programa con una clase que hereda de una clase que no fue definida previamente.
- (IX). Un programa con una clase que herede de más de una clase
- (X). Una declaración de una instancia de una clase, la cual su tipo de dato no sea el mismo con el que se llama al constructor, o una clase superior en el árbol de herencia

6. Ejemplos

Creación de una clase, con su constructor, una variable privada y dos métodos públicos:

```
class myClass {  
    private int a;  
  
    // Constructor  
    public myClass(int a) {  
        this.a = a;  
    }  
  
    public int getA() {  
        return this.a;  
    }  
  
    public int plusOne(int n) {  
        return n + 1;  
    }  
}
```

Instanciación de la clase creada previamente y uso de sus métodos públicos:

```
int main(void) {  
    myClass c1 = new myClass(0);  
    int b = c1.plusOne(c1.getA());  
  
    printf("%d\n", b); // Imprime 1  
  
    return 0;  
}
```

Creación de una clase, sin constructor y dos métodos estáticos:

```
class simpleCalculator {  
    static int sum(int num1, int num2) {  
        return num1 + num2;  
    }  
  
    static int res(int num1, int num2) {  
        return num1 - num2;  
    }  
}
```

Instanciación de la clase creada previamente y uso de sus métodos estáticos:

```
int main(void) {  
    int ans1 = simpleCalculator.sum(2, 8);  
    int ans2 = simpleCalculator.res(ans1, 4);  
  
    printf("%d\n", ans2); // Imprime 6  
  
    return 0;  
}
```

Creación de una clase con un tipo de dato genérico:

```
class GenericAcumulator<T> {  
    private T accumulated;  
  
    // Constructor  
    public GenericAcumulator(T initialValue) {  
        this.accumulated = initialValue;  
    }  
  
    public void accumulate(T value) {  
        this.accumulated = this.accumulated + value;  
        return;  
    }  
  
    public T getAccumulated(void) {  
        return this.accumulated;  
    }  
}
```

Instanciación de la clase creada previamente y uso de sus métodos:

```
int main(void) {  
    GenericAcumulator<int> ga = new GenericAcumulator<int>(0);  
    ga.accumulate(3);  
    ga.accumulate(8);  
  
    printf("%d\n", ga.getAccumulated()); // Imprime 11  
  
    return 0;  
}
```